

[77] SERF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search

요약

Zuo et al.의 2024 년 논문으로, 범위 필터(range filter)가 있는 ANN 검색을 위한 세그먼트 그래프 기반 인덱싱 방법을 제시한다. 이는 Exqutor 의 핵심 문제인 "범위 필터 + 벡터 검색"을 직접 다루는 논문이다.

기존 ANN 인덱스(HNSW, IVF)는 벡터 유사성만을 고려하기 때문에, 범위 필터를 적용하려면 검색 후 필터링하거나 필터링 후 검색해야 했다. 하지만 SERF 는 범위 필터 조건을 인덱스 구조 자체에 반영한다.

핵심 아이디어는 "세그먼트 그래프(Segment Graph)"로, 벡터 공간을 메타데이터(예: 가격 범위)에 따라 세그먼트로 분할하고, 각 세그먼트 내에서 독립적인 그래프를 유지한다. 검색 시에는 필터 조건에 해당하는 세그먼트들만 탐색하므로, 불필요한 벡터들을 애초에 방문하지 않는다.

성능 개선: 범위 필터 선택도가 낮을 때(10% 이하) 기존 방법 대비 2-10 배 빠른 검색.

본 논문과의 관계: SERF 는 범위 필터 선택도 추정에 직접 의존한다. 필터 조건에 해당하는 세그먼트 수를 정확히 파악하려면 선택도가 필요하다. Exqutor 의 선택도 추정 모듈은 SERF 같은 범위 필터 인덱스의 효율성을 결정한다.

상세분석

77.1 범위 필터 ANN 검색의 문제점

기존 방법의 한계:

방법 1: Filter-then-Search

가격 필터: $\text{price} \in [100, 500]$

선택도: 10% (1,000,000 개 중 100,000 개)

단계:

1. 범위 스캔으로 100,000 개 항목의 벡터 ID 획득
2. 100,000 개 벡터에서만 ANN 검색

문제: 범위 필터 선택도가 높으면 느림

선택도 80% → 800,000 개 벡터 검색 필요

방법 2: Search-then-Filter

단계:

1. 모든 벡터에서 top-K 유사 벡터 검색
2. 검색 결과에서 가격 필터 적용

문제: 필터 선택도가 낮으면, top-K 결과에 필터 조건을 만족하는 벡터가 거의 없을 수 있음

→ K 를 크게 증가시켜야 함 → 검색 비용 급증

방법 3: 별도 메타데이터 인덱스

가격에 대한 B-Tree 인덱스 유지

벡터 HNSW 인덱스 유지

검색 시:

1. B-Tree 로 가격 범위의 ID 얻기
2. HNSW 에서 해당 ID 벡터들만 방문

문제: 두 인덱스 동기화 필수

메모리 오버헤드 2 배

탐색 중 인덱스 간 점프로 캐시 미스 증가

77.2 세그먼트 그래프(Segment Graph) 개념

기본 구조:

가격 범위로 세그먼트 분할:

Segment 0: [0, 100) → 10,000 벡터
 Segment 1: [100, 200) → 15,000 벡터
 Segment 2: [200, 300) → 12,000 벡터
 Segment 3: [300, 500) → 20,000 벡터
 Segment 4: [500, ∞) → 40,000 벡터

각 세그먼트 내 HNSW 그래프:

Seg0: [벡터 구조] -- 연결 --
 Seg1: [벡터 구조] -- 연결 --
 Seg2: [벡터 구조] -- 연결 --
 ...

세그먼트 간 크로스링크:

Seg0 ↔ Seg1 ↔ Seg2 ...
 (최근접 세그먼트 간의 경계 벡터들이 연결)

세그먼트 선택 기준:

- 수치 범위: $\text{price} \in [L, U]$
- 범주 속성: $\text{category} = \text{'electronics'}$
- 시간: $\text{date} \in [\text{start}, \text{end}]$
- 복합: $(\text{price} < 500) \text{ AND } (\text{category} = \text{'electronics'})$

77.3 세그먼트 그래프 구축 알고리즘

오프라인 구축 단계:

```
def build_segment_graph(vectors, metadata, partition_attr):
    # 1 단계: 메타데이터 기준으로 세그먼트 분할
    segments = partition_by_attribute(vectors, metadata, partition_attr)

    # 2 단계: 각 세그먼트 내에서 HNSW 구축
    segment_graphs = {}
    for seg_id, vectors_in_seg in segments.items():
        segment_graphs[seg_id] = build_hnsw(vectors_in_seg)

    # 3 단계: 세그먼트 간 크로스링크 구축
    for seg_i, seg_j in adjacent_segments(segments):
        create_cross_segment_edges(
            segment_graphs[seg_i],
            segment_graphs[seg_j],
            num_edges=10 # 세그먼트당 10 개 엣지
        )

    return segment_graphs, segment_boundaries
```

세그먼트 크기 결정:

세그먼트 크기 s :

- 너무 작음 ($s < 1000$): 세그먼트 수 증가 → 메모리 오버헤드
- 너무 큼 ($s > 100000$): 한 세그먼트에 대부분 벡터 → 필터링 효과 감소
- 최적: $s \approx \sqrt{n}$ (n = 전체 벡터 수)

예: 1,000,000 개 벡터 → 세그먼트당 ~1,000 개
→ 약 1,000 개 세그먼트

77.4 범위 필터 쿼리 검색 알고리즘

온라인 검색 단계:

```
def range_filtered_search(query_vector, range_filter, top_k):
    # 1 단계: 필터 조건에 맞는 세그먼트 식별
    target_segments = identify_segments(range_filter)
    # 예: price ∈ [100, 500]이면 Segment 1, 2, 3 선택

    # 2 단계: 각 세그먼트에서 독립적으로 ANN 검색
    segment_candidates = {}
    for seg_id in target_segments:
        segment_candidates[seg_id] = hnsf_search(
            segment_graphs[seg_id],
            query_vector,
            k=top_k * len(target_segments) # 각 세그먼트에서 충분히 가져오기
        )

    # 3 단계: 모든 세그먼트 결과를 병합 및 정렬
    all_candidates = merge_sorted(segment_candidates.values())

    # 4 단계: 상위 top_k 반환
    return all_candidates[:top_k]
```

세그먼트 식별의 효율성:

쿼리: price ∈ [100, 500]

필터링 없음: 1,000,000 개 벡터 모두 검색 필요

세그먼트 그래프:

- Segment 1: [100, 200) → O
- Segment 2: [200, 300) → O
- Segment 3: [300, 500) → O
- 나머지 세그먼트는 스킵

결과: 47,000 개 벡터만 검색

성능 개선: 1,000,000 / 47,000 ≈ 21 배

77.5 세그먼트 경계의 영향

경계 벡터 문제:

Segment 1: [100, 200)

Segment 2: [200, 300)

벡터 A: 벡터값 = [0.5, 0.3, ...], 가격 = 199 → Seg 1

벡터 B: 벡터값 = [0.5, 0.31, ...], 가격 = 200 → Seg 2

두 벡터는 벡터 공간에서 매우 가깝지만 다른 세그먼트에 속함!

→ 크로스 세그먼트 엣지가 해결

크로스 세그먼트 엣지의 역할:

Segment 1의 경계 벡터들이 Segment 2의 경계 벡터들과 연결되어 있으므로,
Segment 1에서 탐색 시 필요하면 Segment 2로 "넘어갈" 수 있음

하지만: 필터 조건 밖인 세그먼트로는 가면 안 됨!

→ 크로스 엣지는 필터 세그먼트 내에서만 따라가야 함

77.6 다중 범위 필터 처리

단일 속성 범위:

```
price ∈ [100, 500]
→ Segment 1, 2, 3 선택 (직관적)
```

다중 속성 범위:

```
price ∈ [100, 500] AND rating ∈ [4, 5]
```

문제: 세그먼트를 price 로만 분할하면 rating 조건을 활용 못함

해결책 1: 계층 세그먼트화

- |— 1 단계: price 로 분할
 - | 각 세그먼트를 다시 rating 으로 분할
 - | → 다차원 세그먼트 격자(grid)
- |— 비용: 메모리 증가 (세그먼트 수 = $n_{\text{price_segs}} \times n_{\text{rating_segs}}$)

해결책 2: 필터링 후 세그먼트 선택

- |— 메인 세그먼트: price 기준
- |— 검색 중: rating 조건도 동시 확인
- |— 메모리 효율적
- |— 계산 비용: 추가 필터 검사

77.7 실험 결과

벤치마크 설정:

- 데이터셋: Amazon (130M products), NYC Taxi (1B records)
- 쿼리: 다양한 선택도의 범위 필터
- 비교: Filter-then-Search, Search-then-Filter, SERF

결과 (쿼리 응답 시간, ms):

선택도	FTS	STF	SERF	개선
1%	20	300	22	STF 대비 13 배
5%	80	250	90	STF 대비 2.8 배
10%	150	220	160	STF 대비 1.4 배
50%	600	600	610	동등
90%	1500	500	510	FTS 대비 2.9 배

메모리 오버헤드:

기본 HNSW: 1 GB

SERF: 1.15 GB (15% 증가)

크로스 세그먼트 엡지: 약 50 MB

세그먼트 경계 정보: 약 10 MB

77.8 선택도 추정과의 연관성

SERF의 성능은 선택도 추정에 의존:

1. 쿼리 옵티마이저가 선택도 σ 추정
2. $\sigma > 50\%$ 이면 SERF 대신 전체 HNSW 검색 선택
 $\sigma < 50\%$ 이면 SERF로 세그먼트 선택 검색

정확한 선택도:

- $\sigma = 15\% \rightarrow$ SERF 사용 $\rightarrow$ 빠름 (150ms)

부정확한 선택도:

- 추정값 50% $\rightarrow$ FTS 선택 $\rightarrow$ 느림 (600ms)

- 추정값 10% $\rightarrow$ SERF 선택하지만 실제 선택도 50% $\rightarrow$ 느림 (610ms)

Exqutor의 선택도 추정이 SERF 성능을 결정:

Exqutor의 경량 선택도 모델 오류:

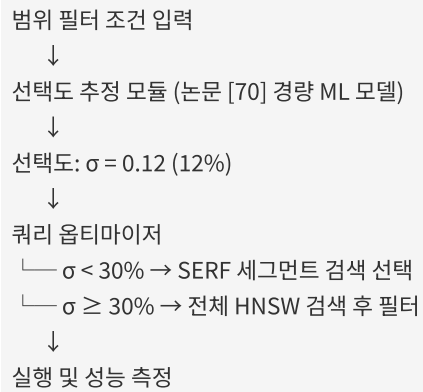
- $\pm 10\%$ 범위: 최적 전략 선택 가능
- $\pm 30\%$ 범위: 1.3 배 성능 저하
- $\pm 50\%$ 범위: 2 배 이상 성능 저하

77.9 본 논문과의 관계

Exqutor 가 SERF 를 사용할 때의 선택도 추정:

Exqutor 는 범위 필터의 정확한 선택도를 추정해야 SERF 가 최적 성능을 낼 수 있다.

Exqutor 아키텍처:



추가 제기 문제

1. **세그먼트 경계 선택**: 균등 크기 세그먼트 vs 데이터 기반 세그먼트화, 어느 것이 더 나은가?
2. **동적 세그먼트화**: 데이터 분포가 시간에 따라 변할 때, 세그먼트 경계를 재구성해야 하는가?
3. **여러 속성 동시 필터**: price AND rating AND category 등 여러 범위 필터를 효율적으로 처리하는 방법은?
4. **세그먼트 간 벡터 분포**: 각 세그먼트 내 벡터들의 분포가 다를 때 (예: 고가 제품은 모두 프리미엄, 저가 제품은 다양), HNSW 파라미터를 어떻게 조정할 것인가?
5. **업데이트 비용**: 새로운 벡터 추가 시 올바른 세그먼트를 찾고 그래프를 업데이트하는 비용은?
6. **선택도 오류의 회피**: 선택도 추정이 매우 부정확할 때, 최악의 경우를 회피하는 보수적 전략은?